Week 3 – Further Development and Testing

Project Area

Web & API Security

Project Title

APIShield – Automated API Security Analyzer

---

1. Introduction

During Week 3, the APIShield project was further developed into a more complete API security assessment tool.

The development focused on implementing security detection modules, risk classification, database and scan management, automated report generation, a web interface, extensive testing, and final security hardening.

Testing was performed continuously against an intentionally vulnerable local API containing fake data. The objective was to verify that APIShield could identify known security weaknesses while maintaining safe and controlled scanning behavior.

---

2. Further Development

Phase 4 – Safe HTTP Client and Scan Engine

The scanning architecture was further developed with a guarded HTTP client and scan engine.

The Safe HTTP Client was designed to ensure that scanner requests remain controlled and safe.

Implemented protections included:

- SSRF validation before requests
- IP validation and pinning

- Redirect validation
- Request timeouts
- Response-size limits
- Request budgets
- Credential isolation
- Cookie isolation
- Protection against unsafe state-changing requests

The scan engine was also developed to execute security checks in a controlled manner.

---

3. Security Detection Modules

Phase 5 – Security Checks and Risk Engine

Security detection modules were implemented for common API security weaknesses.

The implemented checks cover areas such as:

- Authentication
- Authorization
- Broken Object Level Authorization (BOLA)
- Information exposure
- HTTP security headers
- CORS-related weaknesses

A rule-based risk engine was also implemented.

The final project contains:

- **29 security checks**
- **32 risk rules**
- Severity classification
- Finding IDs
- Finding descriptions
- Severity rationale
- Remediation information
- Overall risk assessment

The risk score is explicitly treated as a project-specific heuristic and is **not a CVSS score**.

---

4. Database and Scan Management

Phase 6 – Database, API Routes and Background Scans

The project was extended with persistent scan management using SQLite.

Implemented functionality includes:

- Creating scans
- Tracking scan status
- Storing scan results
- Retrieving scan information
- Deleting scans
- Dashboard statistics
- Background scan execution
- Progress tracking

Additional security validation was implemented for scan requests, including:

- Strict input validation
- SSRF pre-validation
- Request-size restrictions
- Rate limiting
- Concurrent scan limits
- Credential protection
- Parameterized database operations

The scanner was also tested against unsafe targets to verify that dangerous requests were rejected.

---

5. Report Generation and Evaluation

Phase 7 – PDF and JSON Reports

A complete reporting system was implemented.

Both PDF and JSON reports are generated from a shared report data structure to maintain consistency.

The PDF report contains 10 sections:

1. Executive Summary
2. Target Information
3. Tests Performed
4. Findings Summary
5. Detailed Findings
6. Evidence
7. Risk Summary
8. Remediation Recommendations

9. Testing Limitations
10. Report Timestamp

Security testing was also performed on the reporting system.

Scanned content was escaped before being inserted into reports, sensitive information was masked, and report filenames were generated using validated scan IDs.

The report export system also uses atomic file writing and restricts access to the report owner.

---

6. Web Interface Development

Phase 8 – Web Interface

A complete web interface was developed to make APIShield easier to operate.

The interface contains:

Scan Page

- Authorization warning
- Target configuration
- Header and authentication configuration
- Access-control test configuration
- Security-check selection
- Live scan progress
- Result summary

Dashboard

- Total scans
- Total tests
- Passed checks
- Findings by severity
- Risk rating
- Recent scans
- Security charts

Scan History

- Paginated scan history
- Scan results
- Delete functionality
- Confirmation before deletion

Report Page

- All report sections
- Finding severity filters
- PDF download
- JSON download
- Print-friendly layout

The frontend was developed with security and accessibility in mind.

---

7. Security Testing

Extensive automated testing was performed throughout development.

Testing included:

Functional Testing

The project was tested for:

- Scan creation
- Scan execution
- Scan status tracking
- Security finding generation
- Database persistence
- Report generation
- Dashboard functionality
- Scan history
- Report downloads

Security Testing

Security-specific tests covered:

- SSRF protection
- Input validation
- Credential masking
- Host and Origin validation
- Path traversal protection
- XSS/HTML injection handling
- CSP compatibility
- Safe file serving
- Request restrictions
- Secret protection

---

8. Frontend Security Testing

The web interface was tested against hostile input.

Examples included:

- `<script>` payloads
- `<img onerror>` payloads
- `javascript:` links
- HTML markup
- Control characters
- Unicode content

The application uses safe DOM manipulation and avoids dangerous APIs such as:

- `innerHTML`
- `eval`
- `document.cookie`
- `localStorage`

The frontend also follows the application's Content Security Policy requirements.

---

9. Accessibility Testing

Accessibility was included as part of the testing process.

The interface contains:

- Skip navigation link
- Semantic landmarks
- Labelled controls
- Keyboard-accessible controls
- Focusable error summaries
- Live-region announcements
- ARIA attributes
- Text alternatives for charts
- Reduced-motion support
- Forced-colors support
- Print styles

Automated checks were also used to verify WCAG AA color contrast.

---

10. End-to-End Testing

An end-to-end test was performed using the real APIShield application and the intentionally vulnerable API.

The test flow included:

Start APIShield → Start Vulnerable API → Configure Scan → Run Scan → Detect BOLA → Open Report → Download PDF/JSON → Check Dashboard → Delete Scan

The test also verified that sensitive test tokens were not exposed in URLs or generated output.

This provided validation of the complete application workflow rather than testing individual components only.

---

11. Testing Results

Testing progressed continuously during development.

Major milestones included:

Development Stage	Tests Passed
Phase 1	15
Phase 2	34 total
Phase 3	209 total
Phase 4	298 total
Phase 5	419 total
Phase 6	509 total
Phase 7	548 total
Phase 8	589 Python + 54 JavaScript
Final Phase 9	**644 Python + 54 JavaScript**

The final project therefore has:

644 Python tests passing

and

54 JavaScript tests passing.

---

12. Mutation and Security Validation

Mutation-style testing was used during development by intentionally breaking security behaviors and verifying that the test suite detected the changes.

Examples included breaking:

- Credential masking
- Evidence sanitization
- Access-control checks
- CORS protection
- Path validation
- CSP restrictions
- Credential clearing
- Host/Origin validation

These tests helped identify weaknesses in the test suite itself and allowed additional regression tests to be added.

---

13. Final Security Hardening

During final testing, an issue was identified involving requests containing an untrusted `Host` and `Origin`.

The application was updated so that state-changing requests require:

- A trusted Host
- A same-origin Origin

Both protections were intentionally tested by mutation to verify that the security tests could detect their removal.

Another issue was identified with the automatically generated Swagger documentation because it attempted to load scripts from a CDN that conflicted with the application's CSP.

The Swagger UI endpoint was disabled while the OpenAPI specification was retained and documented separately.

---

14. Dependency and Repository Testing

Final project validation also included:

- Dependency vulnerability audit
- Repository secret scanning
- Git repository hygiene checks
- Pinned dependencies
- Removal of unnecessary files and folders
- Automated testing workflow
- Clean installation testing

A fresh environment was used to verify that the project could be installed from the provided dependency file and run successfully.

The dependency audits reported no known vulnerabilities.

---

15. Testing Limitations

Although extensive testing was performed, some limitations remain.

The web interface has not yet been visually verified in a real browser environment. The frontend was tested using automated JavaScript tests, jsdom, accessibility checks, and contrast checks.

Other limitations include:

- Detection is primarily based on patterns and heuristics.
- False positives and false negatives are possible.
- Risk scoring is heuristic and not CVSS.
- Injection vulnerabilities are not covered.
- Business-logic vulnerabilities are not fully assessed.
- Full cross-platform testing on Windows and macOS was not performed.
- CI workflow execution was not performed locally.
- Mutation testing was performed manually rather than as an automated CI stage.

These limitations are documented so that APIShield is presented as a security assessment aid rather than a replacement for a complete penetration test or security review.

---

16. Overall Testing Outcome

The testing and further development stage successfully transformed APIShield into a functional end-to-end API security assessment application.

The final system can:

- Configure authorized API targets
- Perform controlled security scans
- Detect selected API security weaknesses
- Identify BOLA and access-control issues
- Classify findings by severity
- Store and track scan results
- Display findings through a dashboard
- Generate PDF and JSON reports
- Maintain scan history
- Safely handle attacker-controlled content
- Protect credentials and sensitive evidence
- Run automated security and functional tests

---

17. Conclusion

Week 3 focused on further development, testing, security validation, and hardening of APIShield.

The project progressed from an initial scanner implementation into a complete end-to-end API security assessment tool with security checks, risk classification, database support, reporting, web interface, automated testing, and final hardening.

The extensive testing process helped identify and fix several implementation and security issues while improving the reliability of the application.

APIShield is now ready for final documentation, presentation, and release preparation.